

```
In [ ... import marimo as mo
import cv2
import matplotlib.pyplot as plt
```

```
In [ ... img = cv2.imread('a3/spanner.png', cv2.IMREAD_GRAYSCALE)
h, w = img.shape
print(h,w)
```

```
In [ ... ## MEAN FILTERING
def output_img(img,output_dim=305,f_size=3):
    stride = 1
    output_img = []
    for i in range(output_dim):
        row = []
        for j in range(output_dim):
            row.append(0)
        output_img.append(row)
    return output_img
# Mean filter
def mean_filter(img, output_img):
    pixels = img.tolist()
    stride = 1
    f_size = 3 # Filter size
    output_dim = 305
    for r in range(output_dim):
        for c in range(output_dim):

            # Determine where the window starts on the big image
            start_y = r * stride
            start_x = c * stride
            current_sum = 0
            # Loop through the filter window
            for ky in range(f_size):
                for kx in range(f_size):
                    # Get the pixel value
                    i=(start_y + ky)
                    j=(start_x + kx)
                    val = pixels[i][j]
                    current_sum += val

            # Avg find out
            average = current_sum // (f_size * f_size)

            output_img[r][c] = average
    return output_img

empty_out_mean = output_img(img)
mean_filtered = mean_filter(img, empty_out_mean)

plt.imshow(mean_filtered, cmap='gray')
plt.gca()
```

In [...

```
## MEDIAN FILTERING
def median(nums):
    length=len(nums)
    if length%2 == 0:
        median = (nums[length//2]+nums[(length//2)+1])/2
    else:
        median =nums[length//2]

def median_filter(img, output_img):
    pixels = img.tolist()
    stride = 1
    f_size = 3
    output_dim = 305
    for r in range(output_dim):
        for c in range(output_dim):
            # image window on the big image
            start_y = r * stride
            start_x = c * stride
            current_sum = 0
            # list of pixels
            nums = []
            for ky in range(f_size):
                for kx in range(f_size):
                    # Get the pixel value
                    i=(start_y + ky)
                    j=(start_x + kx)
                    val = pixels[i][j]
                    nums.append(val)
            median = median(nums)
            output_img[r][c] = median
    return output_img

empty_out_median = output_img(img)
median_filtered = mean_filter(img, empty_out_median)
plt.imshow(mean_filtered, cmap='gray')
plt.gca()
```

In [...

```
# convert the input into gaussian output
def gaussian_matrix(size,sigma):
    import math
    radius =size//2
    filter = []
    total=0

    ## matrix, inc zeroth pos at centre
    for x in range(-radius, radius+1):
        row = []
        for y in range(-radius, radius+1):
            exp = -(x**2 + y**2)/(2*sigma**2) # power
            val = math.exp(exp)
            row.append(val)
            total += val
        filter.append(row)
```

```

# normalize
for i in range(size):
    for j in range(size):
        filter[i][j] = filter[i][j] / total
return filter

## GAUSSIAN FILTER
def gaussian_filter(img, output_img, filter):
    pixels = img.tolist()
    stride = 1
    f_size = len(filter)
    output_dim = len(output_img)
    for r in range(output_dim):
        for c in range(output_dim):
            # image window on the big image
            start_y = r * stride
            start_x = c * stride
            weighted_sum = 0
            # each pixels in filter
            for ky in range(f_size):
                for kx in range(f_size):
                    # Get the pixel value
                    i=(start_y + ky)
                    j=(start_x + kx)
                    val = pixels[i][j]
                    # apply filter, gives output weight depending on
                    # the ith and j th pos of pixel inside filter
                    wt = filter[ky][kx]
                    weighted_sum +=(val*wt)
            output_img[r][c] = int(weighted_sum)
    return output_img

## Running the filter
# Image_store_dimension
empty_out_gaussian = output_img(img, output_dim=303,f_size=5)
## create the gaussian matrix for out specific filter size and sigma
gaussian_matrix = gaussian_matrix(5,2)

gaussian_filtered_img = gaussian_filter(img, empty_out_gaussian,
gaussian_matrix)

###
plt.imshow(gaussian_filtered_img, cmap='gray')
plt.gca()

```

In [...

```

def sharpen_matrix():
    filter = [
        [0, -1, 0],
        [-1, 5, -1],
        [0, -1, 0]
    ]

```

```

    return filter

def sharpen_filter(img, output_img, filter):
    pixels = img.tolist()
    stride = 1
    f_size = len(filter)

    output_dim = len(output_img)
    for r in range(output_dim):
        for c in range(output_dim):
            # image window on the big image
            start_y = r * stride
            start_x = c * stride
            weighted_sum = 0
            # each pixels in filter
            for ky in range(f_size):
                for kx in range(f_size):
                    # Get the pixel value
                    i=(start_y + ky)
                    j=(start_x + kx)
                    val = pixels[i][j]
                    # apply filter, gives output weight depending on
                    # the ith and j th pos of pixel inside filter
                    wt = filter[ky][kx]
                    weighted_sum +=(val*wt)
            if weighted_sum < 0:
                weighted_sum = 0
            elif weighted_sum >255:
                weighted_sum = 255
            output_img[r][c] = int(weighted_sum)
    return output_img

empty_out_sharpen = output_img(img, output_dim=305,f_size=3)
sharp_matrix = sharpen_matrix()
sharpen_filtered_img = sharpen_filter(img,
empty_out_sharpen,sharp_matrix)

plt.imshow(sharpen_filtered_img, cmap='gray')
plt.gca()

```